LECTURE 4 · A STORY IN FOUR CHAPTERS

PostgreSQL Data Layer, ORM Migrations, OAuth & AI-Assisted Schema Design

The components you made accessible till Session 3 look great — but they're just showing fake data. Today we give them something real to render.

Dr. Mainack Mondal
Dept. of Computer Science and Engineering, IIT Kharagpur
mainack@cse.iitkgp.ac.in

IIT Kharagpur × upGrad

Today's Story, in Four Chapters

Ch.1	Prisma ORM — schema, migrations, CRUD
Ch.2	NextAuth.js OAuth — protected routes
Ch.3	TanStack Query — server state, cache invalidation
Ch.4	AI-assisted schema design — prompting and verifying AI migrations

By the final page, you'll be able to...

Implement a PostgreSQL-backed data layer with Prisma, protect it with OAuth, manage its data on the client with TanStack Query — and know exactly what to check when an AI agent drafts the schema.

The Story So Far

Three sessions in — here's the thread this one picks up.

- Session 1 — AI pair-programming discipline: specify, verify, override
- Session 2 — Next.js App Router, Server Components, strict TypeScript
- Session 3 — accessible components, own your code, verify what AI writes
- Today — those components need real data, real auth, real schemas

> *Same discipline, new surface: verification now applies to database schemas, not just UI.*

How the Four Chapters Fit Together

One request, top to bottom — this is the map every chapter today zooms into a single box

request lifecycle: GET /orders



Chapter 4 (AI-assisted schema design) touches every box above — it's what happens before any of this ships.

Four chapters aren't four separate topics — they're five boxes on one request, taught one at a time.

Handling The Request, Narrated

The same five boxes, read as a sequence of decisions.

- Browser renders /orders — TanStack Query checks its cache first
- Cache miss (or stale): request goes to the Next.js route
- NextAuth.js checks the session before anything else runs
- No session -> redirected to sign in. Session valid -> continue
- Prisma runs a typed, indexed query against PostgreSQL
- Data flows back up; TanStack Query caches it for next time

Notice the order: identity is checked before data is ever touched — that sequencing is the architecture.

Handling The Request, Narrated

The same five boxes, read as a sequence of decisions.

- Browser renders /orders — TanStack Query checks its cache first
- Cache miss (or stale): request goes to the Next.js route
- NextAuth.js checks the session before anything else runs
- No session -> redirected to sign in. Session valid -> continue
- **Prisma runs a typed, indexed query against PostgreSQL**
- Data flows back up; TanStack Query caches it for next time

Notice the order: identity is checked before data is ever touched — that sequencing is the architecture.

Why the Chapters Were Taught in This Order

Not arbitrary — each chapter needed the one before it to make sense.

1. Prisma First

- Nothing to protect or cache until data exists
- Every later chapter assumes a typed schema

2. NextAuth Second

- Now that there's data, decide who can see it
- Sits between the client and Prisma, by design

3. TanStack Query Third

- Now that data is protected, make it feel instant
- Caches what auth already decided the user can have

Chapter 4 (AI schema verification) isn't a fourth layer — it's the review step that should happen before any of the first three ship.

CHAPTER 1

Prisma: Schema, Migrations & CRUD

Zooming into the Prisma box from the map — a GraphQL startup that pivoted its whole business because the real problem was never GraphQL.

How Prisma Came to Be

- 2016 — Graphcool launches: a GraphQL backend-as-a-service
- The team notices the real value wasn't GraphQL delivery speed
- It was that developers stopped managing database workflows by hand
- 2018 — rebrand to Prisma, pivot to database tooling entirely
- 2020 — Prisma 2.0 ships as a standalone, type-safe ORM

Prisma exists because a team building one thing discovered developers actually wanted a different thing.

Wait: What is GraphQL?

GraphQL

- A query language for APIs.
- NO hitting many fixed REST endpoints (/users, /users/1/posts, /posts/1/comments)
- the client sends ONE request describing exactly the shape of data it wants
- The server returns exactly that

Backend-as-a-Service (BaaS)

- A hosted service that gives you a working backend (database + API + auth) without building your own server.
- Think "rented backend" — you configure it, you don't build it.
- "GraphQL backend-as-a-service"
    - a hosted product where you'd define your data model
    - Instantly get a working GraphQL API ontop of it, with no server code of your own.

GraphQL example

a GraphQL query asking for one user's name plus the titles of their posts, nothing else:

```
query {
 user(id: "1") {
  name
  posts {
   title
  }
 }
}
```

Response:

```
{
      "data": {
       "user": {
        "name": "Alice",
        "posts": [
         { "title": "Hello World" },
         { "title": "My Second Post" }
        ]
       }
      }
     }
```

GraphQL Federation: Where to Learn More

Graphcool's original problem, still being solved today — for learners who want to go deeper.

START HERE

What Is GraphQL Federation?

The Guild's vendor-neutral explainer — subgraphs, composition, and a gateway that unifies them.

the-guild.dev/graphql/hive/federation

HANDS-ON

Apollo Voyage: Federation from Day One

A free, interactive course — build an actual federated supergraph step by step.

www.apollographql.com/tutorials/voyage-part1

OPEN ALTERNATIVE

Get Started with Hive Gateway

A fully open-source, Federation-spec-compatible stack — no vendor lock-in to Apollo GraphOS.

the-guild.dev/graphql/hive/docs/schema-registry/get-started/apollo-federation

WHERE IT'S HEADED

The Composite Schemas Working Group

GraphQL Foundation effort (Apollo, Google, Hasura, IBM, The Guild) toward an open federation standard.

graphql.org/blog/2024-05-16-composite-schemas-announcement/

Worth telling learners: *Federation began as one company's 2019 design (Apollo) — the Composite Schemas effort exists because it became the de facto standard anyway. Same pattern as Graphcool.*

How Prisma Came to Be

- 2016 — Graphcool launches: a GraphQL backend-as-a-service
- The team notices the real value wasn't GraphQL delivery speed
- It was that developers stopped managing database workflows by hand
- 2018 — rebrand to Prisma, pivot to database tooling entirely
- 2020 — Prisma 2.0 ships as a standalone, type-safe Object-relational mapper (ORM)

Prisma exists because a team building one thing discovered developers actually wanted a different thing.

Why This Chapter Isn't Optional

This is the goal, stated plainly: hand-written SQL is still one of the most exploited weaknesses on the web.

65%

of web application attacks involve SQL injection (Imperva)

$4.88M

average global data breach cost in 2024 (IBM)

18%

of apps using an ORM are still vulnerable on first scan (2025 industry scan)

January 2025: a SQL injection flaw in PostgreSQL itself (CVE-2025-1094) was used to breach a vendor whose access chain reached the US Treasury Department.

> *An ORM doesn't make this problem go away — it removes the most common way developers accidentally cause it.*

What Is an ORM, and Why Prisma Specifically?

Recall Session 2's strict TypeScript — Prisma's whole pitch is extending that discipline down into your database.

Raw SQL

- Full control, no abstraction
- No compile-time safety
- Manual migration scripts

Prisma

- Schema is the single source of truth
- Generated client is fully typed
- Migrations tracked and versioned

Prisma doesn't replace SQL knowledge — it removes the boilerplate of hand-writing query builders and type definitions.

What the 65% Stat Actually Looks Like in Code

The exact bug class from the last slide — the difference is one line, and it's not obvious which one is dangerous.

```
the vulnerability, and the fix

// String concatenation — user input becomes part of the query
const user = await db.query(
  `SELECT * FROM "User" WHERE email = '${email}'`
);
// email = "' OR '1'='1" returns every row in the table

// Prisma — email is always a bound parameter, never SQL text
const user = await prisma.user.findUnique({
  where: { email },
});
```

> *Prisma's query builder makes the vulnerable version hard to even write by accident — the safety is structural, not a habit you have to remember.*

How to use Prisma: The Scenario

One feature request, walked all the way through the real workflow.

- Product asks for a Comment feature: users can comment on posts
- You'll use Prisma end to end — install, model, migrate, query, ship
- An AI coding agent is available at every step
- The question this deck answers: which steps should it touch, and which shouldn't it?

> *Same task as Chapter 1 — this time we follow it all the way to a deployed migration.*

The Eight Steps, at a Glance

Setup

- 1. Install & initialize
- 2. Point at your database
- 3. Define the schema

Build

- 4. Run the migration
- 5. Use the typed client
- 6. Iterate on changes

Ship

- 7. Browse with Prisma Studio
- 8. Deploy to production

Every step below maps to one of these eight — nothing extra, nothing skipped.

Step 1 — Install and Initialize

Two packages, one command — this scaffolds prisma/schema.prisma and a .env file.

```
terminal

npm install prisma --save-dev
npm install @prisma/client
npx prisma init
```

AI Helps Here

Fine to let an AI agent run this — it's boilerplate scaffolding with nothing project-specific to get wrong.

You Verify This

Just confirm it targeted the right folder if your repo has multiple apps.

Step 2 — Point It at Your Database

```
.env

DATABASE_URL="postgresql://user:password@localhost:5432/mydb"
```

AI Helps Here

AI can write the connection-string format for you.

You Verify This

Never let it fill in real credentials — paste your actual password yourself, outside the chat.

This is the first step with a real secret in it — treat it differently from Step 1.

Step 3 — Define the Schema

The exact prompt from Chapter 1 — and the exact gap it left behind.

prompt to Claude Code

```
Add a Comment model: belongs to a Post, belongs to a
User (author). Include createdAt.
```

AI Helps Here

Drafting the model shape — relations, field types, an initial pass — is exactly what AI is fast at.

You Verify This

Cardinality, indexes, and constraints. This exact prompt shipped without an index on postId — caught in Chapter 4, not by the AI.

This is the highest-leverage step to slow down on — a schema mistake is the hardest of all eight to undo later.

Anatomy of Prisma Schema for User and Post

One file declares the data model, relations, and the generated client target.

prisma/schema.prisma

```
// datasource + generator config omitted — boilerplate,
// same in every project

model User {
  id    String @id @default(cuid())
  email String @unique
  posts Post[]
}

model Post {
  id       String @id @default(cuid())
  title    String
  authorId String
  author   User   @relation(fields: [authorId], references: [id])
}
```

The schema file is the single source of truth — client types, migrations, and documentation all derive from it.

The Reviewed Version

What actually went into schema.prisma, after the review.

prisma/schema.prisma

```
model Comment {
  id        String   @id @default(cuid())
  body      String
  postId    String
  authorId  String
  createdAt DateTime @default(now())
  post      Post     @relation(fields: [postId], references: [id])
  author    User     @relation(fields: [authorId], references: [id])

  @@index([postId])
  @@index([authorId])
}
```

1

Two foreign keys — one relation per parent, exactly as prompted.

2

The fix: indexes on both foreign keys, added during review, not by the AI.

Two lines were the entire difference between the AI's draft and a schema safe to ship.

Step 4 — Run the Migration

```
terminal

npx prisma migrate dev --name add_comment

Applying migration '20260808_add_comment'
  migrations/20260808_add_comment/migration.sql

Your database is now in sync with your schema.
v Generated Prisma Client
```

AI Helps Here

Not really an "AI helps" step — this is a command you run yourself, once the schema is reviewed.

You Verify This

Always read migration.sql before committing it — this is the artifact your reviewer actually sees.

This is the command that makes Step 3's review non-optional — after this runs, the SQL is real.

Step 5 — Use the Typed Client

app/actions/comments.ts

```ts
export function addComment(postId: string, authorId: string, body: string) {
  return prisma.comment.create({ data: { postId, authorId, body } });
}

export function getComments(postId: string) {
  return prisma.comment.findMany({
    where: { postId },
    include: { author: true },
    orderBy: { createdAt: "desc" },
  });
}
```

AI Helps Here

This is AI's strongest step in the whole workflow — CRUD against an already-typed, already-reviewed schema is low-risk and fast.

You Verify This

Just check it queried the field names you actually defined, not ones it assumed.

Notice the shift: Step 3 needed your judgment, Step 5 mostly needs your read-through.

Step 6 — Iterate on Changes

The loop repeats — every future change to the model follows Steps 3 and 4 again.

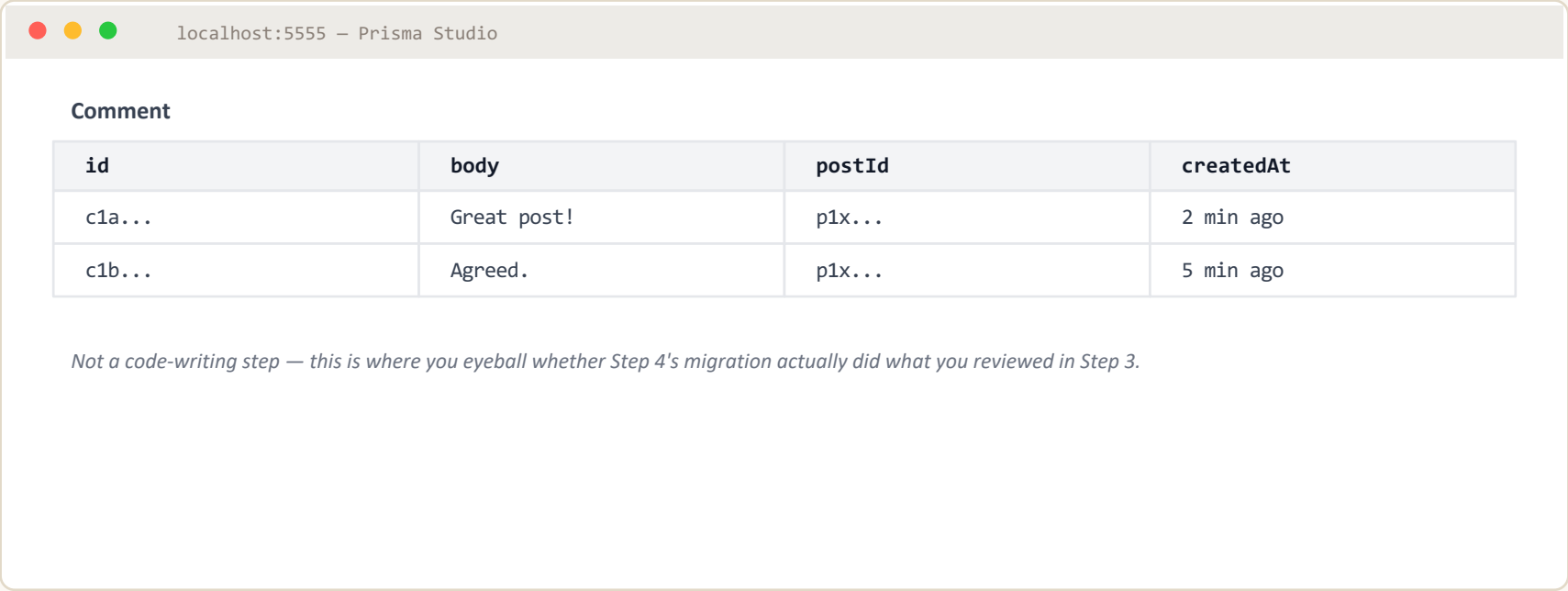
Next Time You Need a Field

- Edit schema.prisma directly
- Run migrate dev --name <change>
- Review the new migration.sql
- Regenerated client picks up the change automatically

> *There's no separate "update" workflow — it's the same edit-review-migrate loop, every time.*

Step 7 — Browse with Prisma Studio

npx prisma studio — a visual data browser, generated automatically from the schema.



Not a code-writing step — this is where you eyeball whether Step 4's migration actually did what you reviewed in Step 3.

AI has no role here — Studio is for a human to look at real rows and sanity-check the result.

What That Schema Looks Like, Rendered

Prisma Studio — a visual data browser generated automatically from the schema on the last slide.



Every model in schema.prisma gets a browsable table here for free — useful for the hands-on investigation

Step 8 — Deploy to Production

```
terminal (CI/CD pipeline)

npx prisma migrate deploy
```

AI Helps Here

None. This is the one command in the entire workflow that should never be run by an autonomous agent.

You Verify This

migrate deploy only applies migrations that already exist — but 'already exist' means someone reviewed them in Step 3.

Chapter 4's rule, restated: a UI bug ships and gets fixed next deploy. A bad migration lives in your data forever.

The Migrations Workflow

Development

- Edit schema.prisma
- npx prisma migrate dev --name add_posts
- Prisma diffs schema, writes SQL, applies it
- Client regenerated automatically

Production

- npx prisma migrate deploy
- Applies pending migrations only — no diffing
- Never runs migrate dev against production

Every migration is a plain SQL file checked into version control — reviewable in the same PR as the schema change.

A Generated Migration

Prisma writes the SQL for you — always read it before merging.

prisma/migrations/20260801_add_posts/migration.sql

```
CREATE TABLE "Post" (
  "id" TEXT NOT NULL,
  "title" TEXT NOT NULL,
  "authorId" TEXT NOT NULL,
  CONSTRAINT "Post_pkey" PRIMARY KEY ("id")
);

ALTER TABLE "Post" ADD CONSTRAINT "Post_authorId_fkey"
  FOREIGN KEY ("authorId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
```

ON DELETE RESTRICT is Prisma's default — decide deliberately whether you want CASCADE, SET NULL, or RESTRICT.

Create, Read, Update, Delete (CRUD) with Prisma Client

app/actions/posts.ts

```
const post = await prisma.post.create({
  data: { title: "Hello", authorId: user.id },
});

const posts = await prisma.post.findMany({
  where: { authorId: user.id },
  include: { author: true },
});

await prisma.post.update({
  where: { id: postId },
  data: { title: "Updated" },
});

await prisma.post.delete({ where: { id: postId } });
```

Every method is fully typed against your schema — a typo in a field name fails at compile time.

Step 8: migrate dev vs db push — Which One?

? Shared team repo with tracked history?	→ **migrate dev**
? Rapid local prototyping, not finalized?	→ **db push**
? About to open a PR with a schema change?	→ **migrate dev — SQL is the reviewable artifact**

RECAP

Where AI Helps, Step by Step

The complete workflow, rated in one glance.

? Step 1-2 — Install, configure	→ **AI is fine, low stakes**
? Step 3 — Define the schema	→ **AI drafts, you verify cardinality + indexes**
? Step 4 — Run the migration	→ **You run it, after review**
? Step 5 — Use the typed client	→ **AI is strong here**
? Step 7 — Prisma Studio	→ **Human-only, visual sanity check**
? Step 8 — Deploy	→ **Never autonomous, always human-triggered**

Prisma vs Alternatives

Feature	Prisma	Drizzle	Raw SQL
Type safety	Generated client	Schema-inferred	Manual
Migration tooling	Built-in, auto-diff	Built-in, explicit SQL	Manual
Learning curve	Low — declarative	Medium — SQL-like	None, no guardrails

Prisma optimizes for velocity and safety; reach for raw SQL only where a query's performance profile demands it.

Chapter 1 Key Terms

Background material worth keeping.

Terms

- ORM — a layer that maps database rows to typed objects in code
- Migration — a versioned, reviewable SQL file that changes the schema
- Prisma Client — the typed query API generated from schema.prisma
- Relation — a foreign-key link between two models, declared in the schema
- Prisma Studio — a visual data browser generated from your schema

END OF CHAPTER 1

Prisma ORM

- schema.prisma is the single source of truth for models, relations, and the client
- migrate dev diffs and applies locally; migrate deploy applies in production only
- Every migration is plain, reviewable SQL checked into version control
- The generated client is fully typed — schema errors are caught at compile time

The takeaway: *Read every generated migration before merging — Prisma writes correct SQL, but 'correct' and 'what you intended' aren't always the same thing.*

Today's Story, in Four Chapters

~~Prisma ORM — schema, migrations, CRUD~~

Ch.2 NextAuth.js OAuth — protected routes

Ch.3 TanStack Query — server state, cache invalidation

Ch.4 AI-assisted schema design — prompting and verifying AI migrations

Up next

NextAuth.js OAuth — the delegated-login problem Twitter had in 2006, and why we still solve it the same way.

CHAPTER 2

NextAuth.js OAuth & Protected Routes

In 2006, a Twitter engineer got tired of apps asking users to hand over their password. That habit still hasn't fully died.

Where does Auth fit?

- Browser renders /orders — TanStack Query checks its cache first
- Cache miss (or stale): request goes to the Next.js route
- **NextAuth.js checks the session before anything else runs**
- **No session -> redirected to sign in. Session valid -> continue**
- Prisma runs a typed, indexed query against PostgreSQL
- Data flows back up; TanStack Query caches it for next time

Notice the order: identity is checked before data is ever touched — that sequencing is the architecture.

How OAuth Came to Be

- Nov 2006 — Twitter's Blaine Cook needs delegated API access
- The norm at the time: hand your password to third-party apps directly
- 2007 — OAuth 1.0 finalized: tokens instead of shared passwords
- 2012 — OAuth 2.0 (RFC 6749) simplifies it with bearer tokens
- Today — the "Sign in with Google" button on nearly every app

> *OAuth exists because password-sharing between apps was once completely normal — and completely insecure.*

Why This Chapter Isn't Optional Either

Same question as Chapter 1, aimed at a different layer: what happens if you roll this yourself and get it wrong?

22%

of all breaches start with stolen or compromised credentials — the #1 initial access vector (Verizon DBIR 2025)

88%

of Basic Web Application attacks involved stolen credentials

292 days

average time to identify a credential-based breach — longer than any other attack type (IBM)

2.8 billion passwords were posted for sale on criminal marketplaces in 2024 alone.

> *Every one of these numbers describes a password your own code was storing and checking — OAuth's whole design is to make that not your problem.*

What Is OAuth 2.0?

Four roles, one goal — your app gets limited access without ever seeing the password.

Resource Owner

- The user
- Owns the account being accessed

Client

- Your Next.js app
- Requests access on the user's behalf

Authorization / Resource Server

- Google, GitHub, etc.
- Issues tokens, verifies identity

OAuth is authorization; OpenID Connect layers authentication on top — NextAuth.js handles both.

QUICK RECAP

What Is OAuth, Again?

One slide of context

The Problem It Solves

- Old pattern: hand your Google password to a third-party app
- OAuth: the app gets limited access, never the password
- Google (the "provider") vouches for who you are

Where NextAuth.js Fits

- A library that implements the OAuth flow for you
- Handles redirects, token exchange, and session creation
- You configure it — you don't hand-roll the protocol

The Authorization Code Flow

Steps 1–3

- 1. User clicks "Sign in with Google"
- 2. App redirects to Google's consent screen
- 3. User approves; Google redirects back with a code

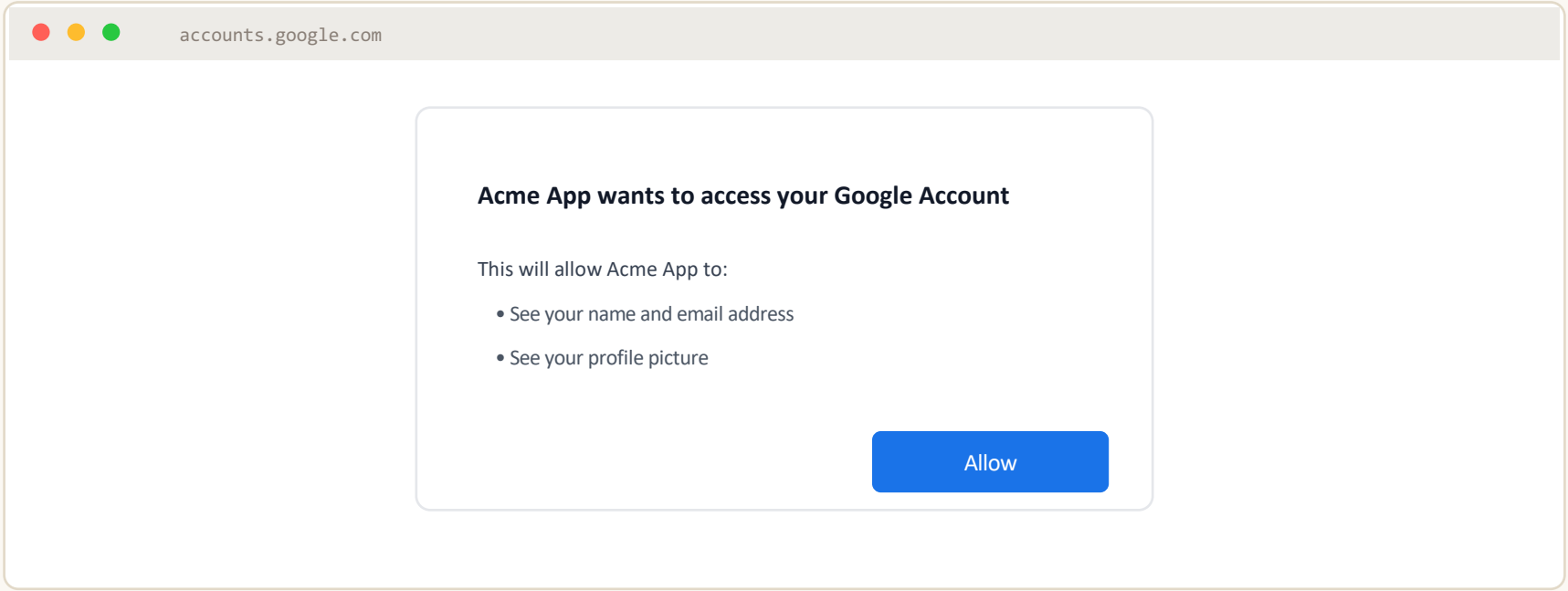
Steps 4–6

- 4. App exchanges the code for tokens (server-to-server)
- 5. Google returns an access token + ID token
- 6. App creates a session for the user

The code-for-token exchange happens server-side, over a channel the browser never sees.

What the Consent Screen Actually Looks Like

Step 2 from the flow above, rendered — this is the moment the user's trust decision happens.



Notice what's absent: no password field. That's the entire point of the protocol.

The Prompt, in Practice

Auth config is exactly the kind of code where a missing line is invisible until it fails.

prompt to Claude Code

```
Set up NextAuth.js with a Google provider and the
Prisma adapter. Use database sessions.
```

Pros
Correct provider config, correct adapter wiring, session strategy set exactly as asked.

Cons
Left NEXTAUTH_URL out of the .env.example file — works locally, silently breaks in production.

Config-shaped bugs like this rarely show up in a demo — they show up during the first real deploy.

NextAuth.js Configuration

auth.ts

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import { PrismaAdapter } from "@auth/prisma-adapter";
import { prisma } from "@/lib/prisma";

export const { handlers, auth } = NextAuth({
  adapter: PrismaAdapter(prisma),
  providers: [Google({
    clientId: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  })],
  session: { strategy: "database" },
});
```

Client secrets live only in environment variables on the server — never in client-side code.

DEFINITIONS

What Is a JWT?

JSON Web Token — an open standard (RFC 7519) for representing claims as a signed, compact string.

What It's For

- A portable way to say "this user is who they claim to be"
- Self-contained — the token itself carries the session data
- No database lookup needed to verify who's asking

What It Is, Literally

- Three parts joined by dots: header.payload.signature
- Each part is Base64URL-encoded JSON
- The whole thing is just a long string of text

A JWT isn't magic — it's JSON, encoded, with a signature stapled on the end.

A JWT, Decoded

The three parts, color-coded — this is what session: { strategy: "jwt" } actually produces.

jwt.io-style decoder

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c3IxIn0.4f_2kP...signature
```

HEADER

```
{ "alg": "HS256", "typ": "JWT" }
```

Which signing algorithm was used

PAYLOAD

```
{ "sub": "usr_1", "email": "a@acme.dev", "exp": 1735689600 }
```

The actual claims — readable by anyone

SIGNATURE

```
HMACSHA256(header + "." + payload, AUTH_SECRET)
```

Proves it wasn't tampered with

> *The payload is readable by anyone who has the token — Base64 is an encoding, not encryption. The signature stops tampering, not reading.*

Session Strategies: JWT vs Database

JWT Sessions

- Session encoded in a signed cookie
- No DB lookup per request — fast
- Hard to revoke before expiry

Database Sessions

- Session ID in cookie, record in Postgres
- Instantly revocable (delete the row)
- One DB read per authenticated request

Default to database sessions for admin/B2B apps where revocation matters; use JWT for high-throughput public APIs.

JWT vs Database Sessions, One More Time

? Need instant revocation (compromised account)?	→ **Database session**
? High-throughput, no per-request DB read?	→ **JWT — this deck's choice**
? Want to inspect exactly what's in the session?	→ **JWT — it's decodable, on purpose**

The Eight Steps, at a Glance

Setup

- 1. Install next-auth
- 2. Register the OAuth app
- 3. Generate a secret, set env vars

Configure

- 4. Write auth.ts
- 5. Add the route handler
- 6. Shape the JWT with callbacks

Ship

- 7. Protect a route + sign-in UI
- 8. Deploy checklist

Step 1 — Install next-auth

```
terminal

npm install next-auth@beta
```

AI Helps Here

Prompt: "Install next-auth v5 (beta) in this Next.js App Router project."

You Verify This

Just confirm it landed as next-auth@5.x, not an accidental v4 install.

@beta currently resolves to Auth.js v5, the App Router-native version this workflow uses.

Step 2 — Register the App with Google

A manual, one-time console step — this is where your Client ID and Client Secret come from.

console.cloud.google.com — Credentials

OAuth 2.0 Client ID

Application type	Web application
Authorized redirect URI	http://localhost:3000/api/auth/callback/google
Client ID	81274...apps.googleusercontent.com
Client secret	GOCSPX-••••••••••••

The redirect URI here must match your app exactly — this is the #1 setup failure.

This step has no code — it's a console form. AI can tell you where to click, but you're the one clicking.

Step 3 — Generate a Secret, Set Env Vars

```
terminal + .env.local

npx auth secret
# writes AUTH_SECRET automatically into .env.local

AUTH_GOOGLE_ID=81274...apps.googleusercontent.com
AUTH_GOOGLE_SECRET=GOCSPX-••••••••••••
```

AI Helps Here

Prompt: "Add placeholder AUTH_GOOGLE_ID and AUTH_GOOGLE_SECRET keys to .env.local, using Auth.js v5's auto-detected naming convention. Leave the values blank for me to fill in."

You Verify This

Never paste the real secret into a chat window. Type it directly into .env.local yourself.

AUTH_SECRET encrypts the JWT and session cookies — treat it exactly like a password.

Step 4 — Write the Config File

auth.ts

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [Google],
  session: { strategy: "jwt" },
});
```

AI Helps Here

Prompt: "Create auth.ts with NextAuth v5. Add a Google provider and set session.strategy to jwt."

You Verify This

Confirm session.strategy matches what you intended — "jwt" vs "database" changes the whole architecture.

One file exports everything: route handlers, the auth() check, and signIn/signOut actions.

Step 5 — Add the Route Handler

app/api/auth/[...nextauth]/route.ts

```
import { handlers } from "@/auth";
export const { GET, POST } = handlers;
```

AI Helps Here

Prompt: "Add the Auth.js route handler at app/api/auth/[...nextauth]/route.ts, exporting GET and POST from auth.ts's handlers."

You Verify This

Just confirm the folder path is exactly [...nextauth] — a typo here breaks every OAuth callback silently.

Two lines. Every sign-in, callback, and sign-out request in your app routes through this file.

Step 6 — Shape What's Inside the JWT

By default the JWT is minimal — callbacks let you add fields, like the user's database id.

auth.ts (extended)

```
export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [Google],
  session: { strategy: "jwt" },
  callbacks: {
    jwt({ token, user }) {
      if (user) token.userId = user.id;
      return token;
    },
    session({ session, token }) {
      session.user.id = token.userId as string;
      return session;
    },
  },
});
```

AI Helps Here

Prompt: "Add jwt and session callbacks to auth.ts so the user's database id ends up on session.user.id."

You Verify This

Double-check what you're adding to the token — anything you put in it is decodable by the browser.

This is the direct payoff of Step 2's mockup — token.userId is now one of the claims you'd see decoded there.

Step 7 — Protect a Route + Sign-In UI

app/orders/page.tsx

```tsx
import { auth, signIn, signOut } from "@/auth";

export default async function OrdersPage() {
  const session = await auth();
  if (!session?.user) {
    return (
      <form action={async () => { "use server"; await signIn("google"); }}>
        <button type="submit">Continue with Google</button>
      </form>
    );
  }
  return <OrdersList userId={session.user.id} />;
}
```

AI Helps Here

Prompt: "Protect app/orders/page.tsx: if there's no session, show a 'Continue with Google' button that calls signIn('google'); otherwise render OrdersList with the session's user id."

You Verify This

Confirm the session check runs on the server, before OrdersList ever renders — not as a client-side redirect after the fact.

auth() works identically here, in middleware, or in a Server Action — one function, everywhere.

Step 8 — Before You Deploy

Must Set in Production

- AUTH_SECRET — different from your local one
- AUTH_GOOGLE_ID / AUTH_GOOGLE_SECRET — production OAuth app

Must Check on the Provider Side

- Add the production redirect URI to Google's console
- https://yourapp.com/api/auth/callback/google, exact match

The #1 production break is a redirect URI that matches locally but was never added for the live domain.

RECAP

Where AI Helps, Step by Step

The complete workflow, rated in one glance.

? Step 1 — Install	→ **AI is fine**
? Step 2 — Register with Google	→ **Manual console step, no code**
? Step 3 — Secret & env vars	→ **AI writes the format, you paste real secrets**
? Step 4-6 — Config, handler, JWT shape	→ **AI is strong — verify strategy + claims**
? Step 7 — Protect a route	→ **AI drafts, you confirm the redirect logic**
? Step 8 — Deploy	→ **Human-checked, every time**

Protecting a Route

app/dashboard/page.tsx

```
import { auth } from "@/auth";
import { redirect } from "next/navigation";

export default async function DashboardPage() {
  const session = await auth();
  if (!session?.user) redirect("/login");

  return <Dashboard userId={session.user.id} />;
}
```

auth() runs on the server — the session check happens before any HTML is sent, the same Server Component pattern from Session 2.

Common OAuth Pitfalls

Configuration

- Redirect URI mismatch between provider console and app
- Client secret committed to version control
- Missing NEXTAUTH_URL in production env

Security

- Trusting client-provided user IDs instead of session data
- Not validating the ID token's issuer and audience

Redirect URI mismatches are the single most common OAuth setup failure — it must match exactly, including trailing slashes.

JWT vs Database Sessions — Which One?

? Need to force-revoke a session instantly?	→ Database sessions
? High-throughput public API, no admin UI?	→ JWT sessions
? Need "sign out other devices" UI?	→ Database sessions

What if you do not trust Google / Meta / Github as your authentication provider?

Would nextauth and jwt still help? Yes, but think through this carefully

The Scenario

Same Orders page, same session strategy — this time nobody is Google.

- Product wants a plain email + password login, no third-party provider
- NextAuth.js's Credentials provider makes this technically possible
- The mechanics are almost identical to the OAuth workflow from before
- The question this deck actually answers: what does 'possible' cost you?

> *This deck shows you exactly how to do it, and exactly what you're signing up for when you do.*

Three Companies That Got This Wrong

Not hypotheticals — three real breaches, three different mistakes, same root cause.

- 2012 — LinkedIn stores 6.5M passwords as unsalted SHA-1 hashes
- Attackers crack hundreds of thousands within days using rainbow tables
- 2013 — Adobe encrypts 153M passwords with one shared key for everyone
- One leaked key decrypts every password in the database at once
- 2018 — Reddit does it right: bcrypt, properly salted — breach exposes nothing usable

Same category of company, same category of breach, wildly different outcomes — the difference was entirely in how passwords were stored.

The Stakes, Restated

The same numbers from the NextAuth.js deck — now aimed at a database you're choosing to own.

22%

of breaches start with stolen or compromised credentials (Verizon DBIR 2025)

$4.81M

average cost of a credential-based breach (IBM)

292 days

average time to even detect one — longer than any other breach type

With Google OAuth, this risk sits on Google's infrastructure. With Credentials, it sits on yours.

> *Nothing about these numbers changes because your app is small — attackers don't check your user count first.*

What OAuth Was Quietly Doing for You

None of this is optional once you own the login form — it's just invisible until you do.

Google Handled This

- Verifying the password was correct
- Storing credentials securely
- "Forgot password" flow
- Detecting suspicious logins
- Email verification

Now You Handle This

- Hashing algorithm choice (and getting it right)
- Building your own reset flow, safely
- Rate limiting brute-force attempts
- Account lockout policy
- Every one of these becomes a security surface

> *This is the actual trade you're making — not "OAuth vs. no OAuth," but ”a provider’s security team vs. yours."*

If You're Doing This Anyway — Do It Properly

Five steps, each one non-negotiable if you skip Step 1 of this whole deck: using OAuth instead.

Build

- 1. Add the Credentials provider
- 2. Hash passwords correctly
- 3. Write authorize()

Defend

- 4. Rate-limit login attempts
- 5. Accept what you still don't have

Step 1 — Add the Credentials Provider

The shape is nearly identical to the Google provider from the last deck.

auth.ts

```ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Credentials({
      credentials: { email: {}, password: {} },
      async authorize() {
        return null; // real check comes in Step 3
      },
    }),
  ],
  session: { strategy: "jwt" },
});
```

AI Helps Here

Prompt: "Add a NextAuth Credentials provider with email and password fields. Leave authorize() returning null for now — I'll implement the real check next."

You Verify This

This part is genuinely low-risk — it's Step 3's authorize() function where the real danger lives.

Step 2 — Hash Passwords Correctly

This is LinkedIn and Adobe's mistake, made preventable in three lines.

lib/password.ts

```ts
import bcrypt from "bcryptjs";

export async function hashPassword(plain: string) {
  return bcrypt.hash(plain, 12); // 12 salt rounds
}

export async function verifyPassword(plain: string, hash: string) {
  return bcrypt.compare(plain, hash);
}
```

AI Helps Here

Prompt: "Write hashPassword and verifyPassword using bcryptjs with 12 salt rounds. Never use MD5, SHA-1, or plain encryption for this."

You Verify This

Confirm it's bcrypt or argon2 — an AI that's seen enough old tutorials may default to MD5/SHA-1, LinkedIn's exact mistake.

bcrypt salts automatically, per-password — there is no shared key to leak, unlike Adobe's breach.

Step 3 — Write authorize()

The highest-stakes function in this entire deck — this is the login check itself.

```
auth.ts (verifyLogin)

Credentials({
  credentials: { email: {}, password: {} },
  async authorize(creds) {
    const user = await prisma.user.findUnique({
      where: { email: creds.email },
    });
    if (!user) return null;

    const valid = await verifyPassword(creds.password, user.hashedPassword);
    if (!valid) return null;

    return { id: user.id, email: user.email };
  },
}),
```

AI Helps Here

Prompt: "Replace the placeholder authorize() from Step 1 with a real check: look up the user by email, verify the password with verifyPassword, return null on any failure."

You Verify This

Read every line yourself. A subtle bug here (e.g. returning a user object on a failed check) is a full authentication bypass.

> *this function is the actual lock on the door.*

Step 4 — Rate-Limit Login Attempts

auth.ts (guarded)

```
async authorize(creds) {
  const attempts = await getRecentAttempts(creds.email);
  if (attempts >= 5) throw new Error("Too many attempts");

  const user = await prisma.user.findUnique({
    where: { email: creds.email },
  });
  const valid =
    user && (await verifyPassword(creds.password, user.hashedPassword));

  await recordAttempt(creds.email, !!valid);
  return valid ? { id: user.id, email: user.email } : null;
},
```

AI Helps Here

Prompt: "Add rate limiting to the authorize() function from Step 3 — max 5 attempts per email in a rolling window, using a getRecentAttempts/recordAttempt helper."

You Verify This

Decide the storage yourself (Redis, DB table) — and confirm it limits by email AND by IP, not just one.

This step doesn't exist in the OAuth version of this workflow — Google already solved it, invisibly.

Step 5 —What You Still Don't Have

Not Built Yet

- Password reset flow (and its own security surface)
- Email verification on signup
- Account lockout / suspicious-login detection
- Two-factor authentication
- Compliance with any password-strength policy

Every item on this list was free with Google OAuth. Every item here is now homework — and each one can itself be built insecurely.

Is Custom Auth Ever Justified?

? An internal tool, no OAuth provider available at all?	→ **Reasonable — proceed carefully**
? A product requirement mandates it, in writing?	→ **Justified — build it properly**
? "OAuth felt like extra setup"?	→ **Not a reason — use OAuth**
? "We want full control over the login UI"?	→ **Not a reason — Credentials provider still uses NextAuth's UI freely; OAuth doesn't limit this**

Chapter 2 Key Terms

Background material worth keeping — plus one honest heads-up about where this ecosystem is headed.

Terms

- OAuth — an authorization framework; the app gets access, never the password
- Access token — short-lived credential sent with each API request
- Refresh token — long-lived, used to obtain new access tokens
- Session strategy — how login state is stored: JWT cookie or database row

Worth Knowing

- NextAuth.js is now part of a broader project called Auth.js
- Auth.js itself now points new projects toward Better Auth for some use cases
- Everything taught here still works — just know the ecosystem is moving

END OF CHAPTER 2

NextAuth.js OAuth & Protected Routes

- ✓ OAuth 2.0 delegates authentication to a trusted provider — your app never sees the password
- ✓ The authorization code exchange happens server-to-server, never exposed to the browser
- ✓ Choose JWT sessions for stateless throughput, database sessions when you need revocation
- ✓ Protect routes by checking the session in Server Components, before rendering

The takeaway: *Authentication is a solved problem — delegate to NextAuth.js and a trusted provider rather than rolling your own.*

Today's Story, in Four Chapters

~~Prisma ORM — schema, migrations, CRUD~~

~~NextAuth.js OAuth — protected routes~~

Ch.3 TanStack Query — server state, cache invalidation

Ch.4 AI-assisted schema design — prompting and verifying AI migrations

Up next

TanStack Query — born from a 2019 conversation about a React community drowning in Redux.

CHAPTER 3

TanStack Query: Server State & Cache Invalidation

One conversation between two developers in 2019 talked an entire community out of putting API responses in Redux.

Last piece of the puzzle: Cache and DB

- Browser renders /orders — TanStack Query checks its cache first
- Cache miss (or stale): request goes to the Next.js route
- NextAuth.js checks the session before anything else runs
- No session -> redirected to sign in. Session valid -> continue
- Prisma runs a typed, indexed query against PostgreSQL
- Data flows back up; TanStack Query caches it for next time

Notice the order: identity is checked before data is ever touched — that sequencing is the architecture.

How TanStack Query Came to Be

- 2019 — the React community puts everything into Redux's global store
- Tanner Linsley and Kent C. Dodds talk through why that's the wrong model
- Server data isn't "your" state — it's a snapshot that expires
- Dec 2019 — Tanner ships React Query around that one insight
- 2022 — rebranded TanStack Query as it expands beyond React

> *The core idea was simple enough to fit in a conversation: a **cache**, not a reducer.*

Learning TanStack – lets build something

Meet PawMatch

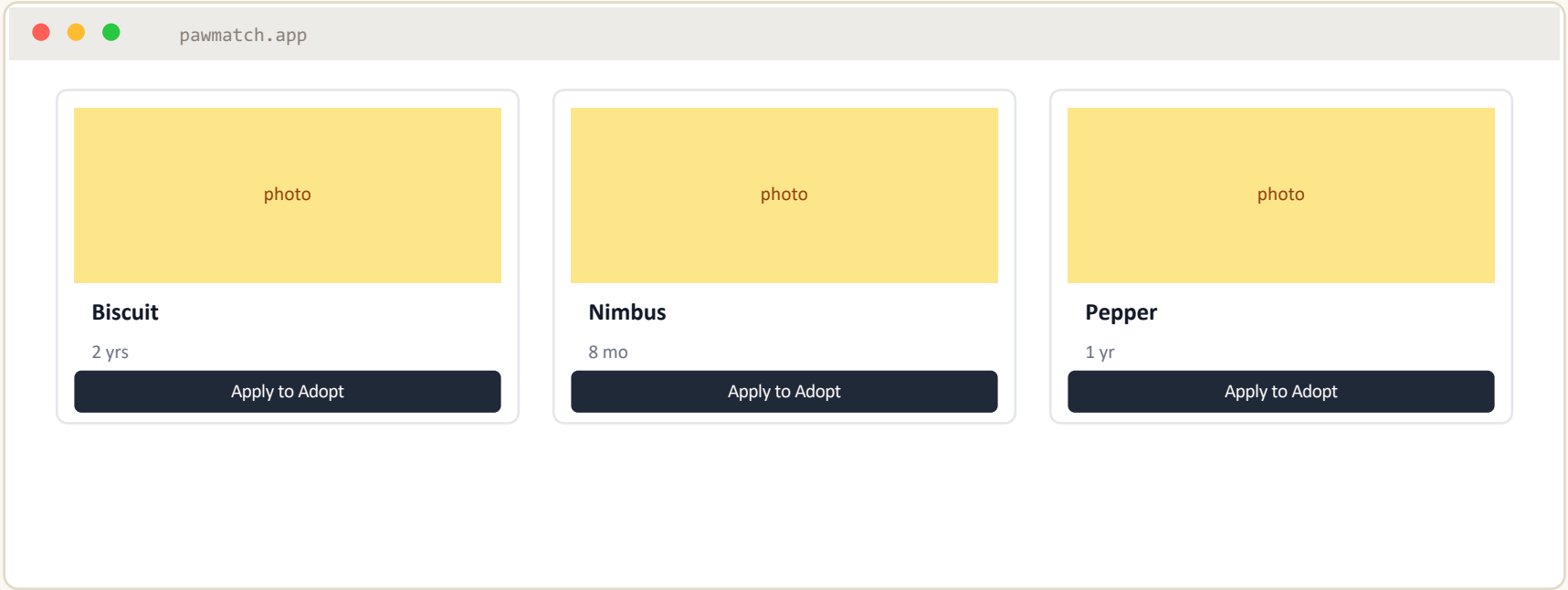
A small app we'll build together, one small piece at a time.

- PawMatch shows a list of puppies available for adoption
- Each puppy has a photo, a name, and a heart button to save favorites
- You can click "Apply to Adopt" on any puppy
- You can switch between a grid view and a list view

> *Every technical idea in this deck exists because of one of these four features — we'll meet each idea only when we actually need it.*

What PawMatch Looks Like

Keep this picture in mind — every slide from here on is building one piece of this screen.



A photo grid, a heart button, an "Apply" button, and a view toggle — that's the whole app.

What Is "State"?

Our very first term — everything else in this deck builds on this one word.

In Plain Words

- "State" just means: data your app is currently showing or remembering
- The list of puppies is state
- Whether you've favorited a puppy is state
- Whether you're viewing a grid or a list is state

> *If a screen can look different at different times, whatever decides that difference is "state."*

Two Kinds of State — Where Does the Data Live?

This one distinction explains almost every decision in this deck.

Server State

- Data that lives on a server, somewhere else
- Example: the list of puppies — stored in PawMatch's database
- Can change without you doing anything (someone else adds a puppy)

Client State

- Data that only exists in your browser, right now
- Example: "grid view" vs "list view"
- Nobody else needs to know about it; it's yours alone

> *Server state is borrowed. Client state is owned. That difference is why we'll end up using two different tools.*

Server State Is Not Client State

The distinction the 2019 conversation was actually about.

Client State

- Form input values
- UI toggles (modal open/closed)
- You fully own and control it

Server State

- User records, posts, orders
- Can go stale the moment it's fetched
- Shared across components and tabs

TanStack Query treats every fetch as a cache entry with a lifecycle — fetching, stale, refetching — instead of a one-off effect.

The reason you need to know this

The useEffect-Fetch Pattern

- Every component re-fetches on its own — no shared cache
- No standard for loading/error states — each dev invents one
- Race conditions when a component unmounts mid-fetch
- Stale data with no way to know it's stale

What This Costs a Team

- Duplicate network requests for the same data
- UI showing different "truths" in different components
- Redux used to paper over the gap, badly
- Kent C. Dodds: "If React Query had existed, Redux wouldn't have been nearly as popular"

This is what the 2019 conversation from the last slide was actually solving — not a missing feature, a missing model.

TanStack Query Handles Server State — What About Client State?

Worth naming directly: this curriculum pairs TanStack Query with Zustand for exactly this reason.

TanStack Query

- Server state: orders, posts, users
- Owns fetching, caching, invalidation
- This chapter's entire focus

Zustand

- Client state: theme, sidebar open, wizard step
- A small, unopinionated store — no boilerplate
- Not covered in depth today — but this is where it fits

Neither tool tries to do the other's job — that division is exactly why the two pair well, and why teams that use one for both end up fighting it.

Part 1 of app

Showing the Puppies

Our first job: get the puppy list from the server onto the screen.

The Old Way — Before Any Special Tool

Let's see what this looks like without help, so the next tool's value is obvious.

What You'd Have to Write Yourself

- A variable to hold the puppy list
- A variable for "is it still loading?"
- A variable for "did something go wrong?"
- Code to re-fetch if the data might be stale
- Code to avoid fetching the same thing twice

None of this is puppy-specific — every app that shows server data needs all five of these, every time.

What Is TanStack Query?

Our first named tool — introduced only now that we've felt the problem it solves.

In Plain Words

- A helper library for fetching server data in React apps
- It writes the five things from the last slide, so you don't
- It also remembers what it fetched, so it doesn't ask twice unnecessarily

> *Think of it as a well-organized assistant: "I'll go get the puppy list, remember it, and tell you if it's out of date."*

What Is a "Hook" in React?

*TanStack Query's main tool is called **useQuery** — before that makes sense, we need this word.*

In Plain Words

- A hook is a special function whose name starts with "use"
- It lets a component "hook into" a React feature
- You've likely already seen useState — this is the same family
- **useQuery** is just a hook made specifically for fetching data from server in TanStack

Any time you see a function called use-something, it's a hook — a signal that it connects to something React manages for you.

What Is a "Cache"?

One more word before we touch code — TanStack Query's whole job centers on this.

In Plain Words

- A cache is a temporary "remember this" storage spot
- Instead of asking the server every single time, you check the cache first
- If the cache has a recent enough answer, you use that — instantly
- If not, you ask the server again and update the cache

> *A cache is like keeping a photo of the fridge's contents on your phone, so you don't open the fridge every five minutes.*

Step 1 — Install TanStack Query

Our first actual command.

```
terminal

npm install @tanstack/react-query
```

AI Helps Here

Prompt: "Install @tanstack/react-query in this React project."

You Verify This

A single, safe install — nothing project-specific to double-check yet.

What Is a "Provider" in React?

One more term — the very next step needs it.

In Plain Words

- A Provider is a wrapper component
- You put it near the top of your app, around everything else
- Everything inside it can now use whatever the Provider is sharing
- TanStack Query needs one, so its cache is available everywhere

> *Think of a Provider like the Wi-Fi router in a house — put it in one place, and every room gets the signal.*

Step 2 — Set Up the Provider

This file's only job is to turn on the "Wi-Fi router" from the last slide.

app/providers.tsx

```tsx
import { useState } from "react";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

export function Providers({ children }) {
  const [client] = useState(() => new QueryClient());
  return (
    <QueryClientProvider client={client}>
      {children}
    </QueryClientProvider>
  );
}
```

AI Helps Here

Prompt: "Create a Providers component wrapping the app in a TanStack QueryClientProvider. Create the QueryClient with useState."

You Verify This

Confirm it used useState(() => new QueryClient()) — we'll see exactly why on the next slide.

A Small Detail Worth Pausing On

Why useState, and not just a plain variable?

The Short Version

- Your app might run on a shared server, serving many different visitors
- If the cache is created as a plain, shared variable, everyone accidentally shares one cache
- One visitor's puppy data could leak into another visitor's screen
- useState makes sure each visitor quietly gets their own fresh cache

This is the first "folly" in this deck — a mistake real teams have shipped by accident. We'll see the broken version shortly.

The Broken Version, So You Can Recognize It

Looks almost identical to Step 2 — that's exactly why it's dangerous.

app/providers.tsx (the folly)

```tsx
import { QueryClientProvider, QueryClient } from "@tanstack/react-query";

const client = new QueryClient(); // created once, shared by everyone

export function Providers({ children }) {
  return (
    <QueryClientProvider client={client}>
      {children}
    </QueryClientProvider>
  );
}
```

> *One line moved outside the component — and now every visitor to the site shares the exact same remembered data.*

Meet useQuery

The hook that actually fetches the puppies — now that Provider and cache both make sense.

What You Give It

- A name for what you're fetching (so it can be remembered by that name)
- A function that goes and gets the data

What It Gives You Back

- The data itself, once it arrives
- A true/false flag for "is this still loading"
- Everything updates automatically when the cache changes

> *You describe what you want; useQuery handles the loading flag, the caching, and the re-fetching, all by itself.*

Step 3 — Fetch the Puppy List

app/puppy-list.tsx

```tsx
import { useQuery } from "@tanstack/react-query";

function PuppyList() {
  const { data, isLoading } = useQuery({
    queryKey: ["puppies"],
    queryFn: () => fetch("/api/puppies").then((r) => r.json()),
  });

  if (isLoading) return <p>Loading puppies...</p>;
  return (
    <div>
      {data.map((puppy) => <PuppyCard key={puppy.id} puppy={puppy} />)}
    </div>
  );
}
```

AI Helps Here

Prompt: "Fetch the puppy list with useQuery from /api/puppies. Show a loading message while it's loading, then render a PuppyCard for each puppy."

You Verify This

Check the queryKey — ["puppies"] is the name this data is remembered under. Get it wrong and caching gets confusing fast.

This one hook replaces every piece of manual bookkeeping from the "Old Way" slide, several slides ago.

useQuery vs useState — Which One?

? Data originates from your server / database?	→ useQuery
? Purely local UI state (modal, input value)?	→ useState
? Multiple components need the same fetched data?	→ useQuery — shared cache

Part 2 of app

Applying to Adopt a Puppy

So far we've only been reading data. Now we need to change something.

The Need

A very ordinary click, with a very ordinary expectation.

- You click "Apply to Adopt" on Biscuit's card
- You expect: an application gets created on the server, somewhere
- This isn't fetching data anymore — it's changing data
- useQuery was built for fetching. We need a different tool for changing.

> *Reading data and changing data are different enough problems that TanStack Query gives them two different hooks.*

What Is a "Mutation"?

A new word for a familiar idea.

In Plain Words

- A mutation is any action that changes data on the server
- Creating something new (an adoption application)
- Updating something (editing a puppy's bio)
- Deleting something (cancelling an application)
- Anything that isn't just "go get me data" is a mutation

> *If the action would need a POST, PUT, PATCH, or DELETE on a typical web API, it's a mutation.*

Meet useMutation

TanStack Query's tool for changes, sitting right next to useQuery.

What You Give It

- A function describing the change to make (e.g. "send this application")

What You Get Back

- A mutate function you call whenever the user clicks the button
- A way to run code "on success" — useful very soon

useQuery runs automatically when the component appears. useMutation waits — it only runs when you tell it to.

Step 4 — Write the Apply-to-Adopt Mutation

app/apply-button.tsx

```tsx
import { useMutation } from "@tanstack/react-query";

function ApplyButton({ puppyId }) {
  const mutation = useMutation({
    mutationFn: () =>
      fetch(`/api/applications`, {
        method: "POST",
        body: JSON.stringify({ puppyId }),
      }),
  });

  return (
    <button onClick={() => mutation.mutate()}>Apply to Adopt</button>
  );
}
```

AI Helps Here

Prompt: "Add a useMutation that POSTs to /api/applications with the puppyId, triggered by clicking Apply to Adopt."

You Verify This

This sends the application, but doesn't yet update anything on screen — that's the next problem.

The Need, Again

Click the button. The application is sent. And yet...

- You click "Apply to Adopt" on Biscuit
- The server successfully saves the application
- You look at the screen — Biscuit's card still says "Apply to Adopt"
- Nothing visually confirms it worked, because the puppy list is still the old, cached data

> *Remember the cache from earlier? It's doing exactly its job — remembering the old answer. That's now the problem.*

What Is "Cache Invalidation"?

The fix for the exact problem on the last slide.

In Plain Words

- "Invalidating" a cache entry means marking it as no longer trustworthy
- It doesn't delete the old data immediately
- It tells TanStack Query: "next chance you get, go check again"
- Any component using that data automatically re-fetches

It's the difference between throwing away yesterday's newspaper and just writing "OUTDATED" across the front page.

Step 5 — Invalidate the Puppy List After Applying

app/apply-button.tsx (updated)

```tsx
import { useMutation, useQueryClient } from "@tanstack/react-query";

function ApplyButton({ puppyId }) {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: () =>
      fetch("/api/applications", {
        method: "POST",
        body: JSON.stringify({ puppyId }),
      }),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["puppies"] });
    },
  });

  return (
    <button onClick={() => mutation.mutate()}>
      Apply to Adopt
    </button>
  );
}
```

AI Helps Here

Prompt: "After the mutation succeeds, invalidate the ['puppies'] query so the list refetches and shows the updated status."

You Verify This

Notice the queryKey here — ["puppies"] — matches the queryKey from Step 3 exactly. That match is how TanStack Query knows what to refresh.

Now the moment the server confirms the application, the puppy list quietly refetches and Biscuit's card updates.

Part 3 of app

The Heart Button

One more small feature, and one more small problem to notice.

The Need

A tiny interaction, and a very human expectation about it.

- You tap the heart icon on Nimbus's card to favorite him
- Using Step 4 and 5's pattern, this is technically "just another mutation"
- But there's a brief pause before the heart turns red — the network round trip
- On a slow connection, that pause feels laggy, even though nothing is actually wrong

> *The mutation pattern from Part 2 is correct here too — but for this specific interaction, correct isn't the same as feeling good.*

What Is an "Optimistic Update"?

A new idea for exactly this situation.

In Plain Words

- Update the screen immediately, before the server has replied
- Assume — "optimistically" — that the request will succeed
- If it does succeed, nothing else needs to happen
- If it fails, quietly put the screen back the way it was

The heart turns red the instant you tap it — the network request is still happening in the background, invisibly.

The Four Steps of an Optimistic Update

In plain language, before any code — this is the recipe we're about to follow.

1. Snapshot

- Before changing anything, remember what the screen looked like

2. Update

- Immediately change the screen to the new, hoped-for state

3. Send

- Send the real request to the server, same as any mutation

4. Fix if Needed

- If it failed, put back the snapshot from step 1

Step 1 exists purely so step 4 is possible — you can't undo a change you didn't first write down.

Step 6 — Make the Heart Button Optimistic

app/favorite-button.tsx

```tsx
const mutation = useMutation({
  mutationFn: () =>
    fetch(`/api/favorites/${puppyId}`, { method: "POST" }),

  // 1. Snapshot, 2. Update
  onMutate: async () => {
    const previous = queryClient.getQueryData(["puppies"]);
    queryClient.setQueryData(["puppies"], (old) =>
      old.map((p) =>
        p.id === puppyId ? { ...p, favorited: true } : p
      )
    );
    return { previous };
  },

  // 4. Fix if needed
  onError: (err, vars, context) => {
    queryClient.setQueryData(["puppies"], context.previous);
  },
});
```

AI Helps Here

Prompt: "Make the favorite mutation optimistic: snapshot the puppies query in onMutate, update the heart immediately, and roll back in onError."

You Verify This

Check that onError actually restores context.previous — an optimistic update with no rollback path just quietly lies to the user on failure.

Step 3 (Send) is just mutationFn itself, already running the whole time — nothing extra to write for it.

Final part of app

Remembering the View Preference

Grid view or list view — a very different kind of problem from anything so far.

The Need

A small toggle, that turns out not to fit anywhere we've built so far.

- PawMatch should let you switch between a grid view and a list view
- This choice isn't stored on any server — nobody else needs to know it
- It's not fetched, so useQuery doesn't apply
- It's not sent anywhere, so useMutation doesn't apply either

> *This is exactly the "client state" category from the very first slides — time to meet the tool built for it.*

Could We Just Use useState?

Worth asking honestly before reaching for a new tool.

Where useState Works Fine

- The view toggle lives in exactly one component
- Nothing else in the app needs to know about it

Where It Starts to Struggle

- The header needs to show the current view
- The puppy grid needs to know the current view too
- Now the state has to be passed down through components that don't otherwise need it

That last problem has a name, and it's annoying enough that it earns its own slide next.

What Is "Prop Drilling"?

The specific annoyance from the last slide, named.

In Plain Words

- "Props" are values passed from a parent component to a child
- "Drilling" happens when you pass a prop through several components
- ...just so it can reach one deeply nested component that actually needs it
- The components in between don't use it — they just relay it

> *Imagine passing a note through five people in a line, and only the fifth person actually needed to read it.*

What About React's "Context"?

React actually has a built-in answer to prop drilling — worth knowing, and knowing its limit.

What Context Does

- Lets you skip the relay — any component can read the value directly
- Solves prop drilling completely

Its Trade-Off

- Every component reading that Context re-renders on any change
- For something that changes often, this gets expensive fast

Context solves one problem and quietly introduces a performance one — which is exactly the gap Zustand was built to close.

What Is Zustand?

Our second and final named tool.

In Plain Words

- A small library for sharing client state across components
- No prop drilling — any component can use it directly
- No Context — so it avoids Context's re-render cost
- "Zustand" is German for "state" — a fittingly literal name

> *Where TanStack Query remembers server answers, Zustand remembers your app's own local decisions.*

Step 7 — Install Zustand and Create a Store

A "store" is just what Zustand calls the place it keeps your shared state.

terminal + lib/view-store.ts

```
npm install zustand

import { create } from "zustand";

export const useViewStore = create((set) => ({
  view: "grid",
  setView: (newView) => set({ view: newView }),
}));
```

AI Helps Here

Prompt: "Install zustand. Create a store with a view field (starting as 'grid') and a setView action."

You Verify This

No Provider needed here — unlike TanStack Query, Zustand doesn't wrap your app in anything.

Compare this to Step 2's Provider setup — Zustand deliberately skips that entire step.

Step 8 — Use It in the Toggle Button

Any component, anywhere, can now read and change the view — no passing props required.

app/view-toggle.tsx

```tsx
import { useViewStore } from "@/lib/view-store";

function ViewToggle() {
  const { view, setView } = useViewStore();
  const next = view === "grid" ? "list" : "grid";

  return (
    <button onClick={() => setView(next)}>
      Switch to {next} view
    </button>
  );
}
```

AI Helps Here

Prompt: "Build a ViewToggle button that reads view and setView from useViewStore and switches between grid and list."

You Verify This

No prop drilling to check for — that absence is the entire point of using Zustand here.

The Folly — Mixing Up the Two Tools

The single most common mistake once both tools are in a project.

Puppies Stored in Zustand?

- No automatic refetching
- No cache invalidation after a mutation
- Two different "sources of truth" if TanStack Query is used elsewhere too

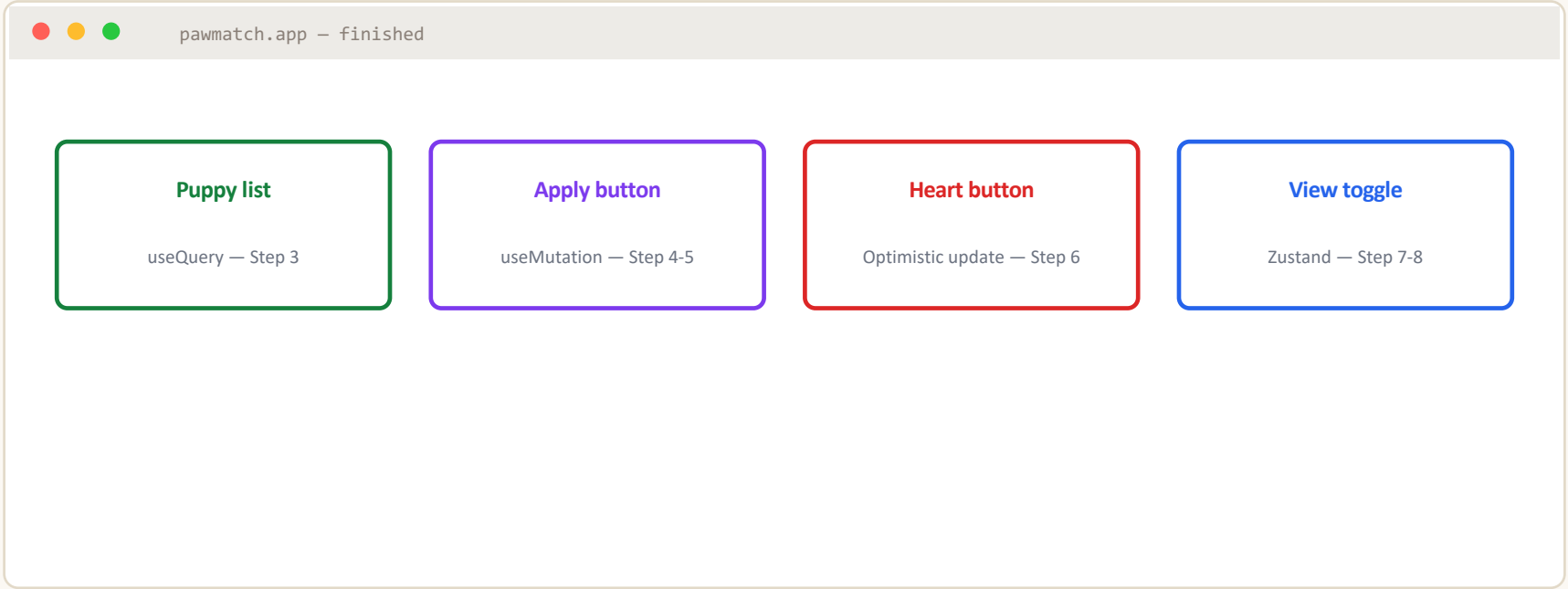
View Preference Stored in TanStack Query?

- A queryFn that never actually fetches anything
- Loading and error states that can never really happen
- Confusing for the next person reading the code

The fix is the same rule from the very first slides: if it comes from a server, it's TanStack Query's job; if it doesn't, it's Zustand's.

PawMatch, Complete

Every box below traces back to one specific step in this deck.



Four small features, four small ideas, introduced only exactly when each one was needed.

Which Tool for Which Job?

? Reading data that lives on a server?	→ **useQuery**
? Creating, updating, or deleting something on a server?	→ **useMutation**
? Needs to feel instant, before the server replies?	→ **Optimistic update**
? State that's only ever local to the browser?	→ **Zustand**

Chapter 3 Key Terms

Background material worth keeping.

Terms

- Server state — data owned by a server, cached and kept fresh, not by you
- queryKey — the identifier a cache entry is stored and invalidated under
- staleTime — how long cached data is treated as fresh before refetching
- Optimistic update — updating the UI before the server confirms, with rollback on failure
- Cache invalidation — marking a cache entry stale so it refetches

END OF CHAPTER 3

TanStack Query

- Server state has a lifecycle — fresh, stale, refetching — that useState doesn't model
- queryKey identifies a cache entry; staleTime controls how long it's considered fresh
- invalidateQueries after a mutation is how you keep server state and UI in sync
- Optimistic updates need cancel, snapshot, write, rollback-on-error, resync

The takeaway: *Stop fetching data in useEffect — let TanStack Query own the cache lifecycle so you don't have to hand-roll it.*

Today's Story, in Four Chapters

- ~~Prisma ORM — schema, migrations, CRUD~~
- ~~NextAuth.js OAuth — protected routes~~
- ~~TanStack Query — server state, cache invalidation~~
- **Ch.4** AI-assisted schema design — prompting and verifying AI migrations

Up next

AI-assisted schema design — the highest-consequence artifact an AI agent can touch this session, and why it needs the most scrutiny.

CHAPTER 4

AI-Assisted Schema Design

Prisma doesn't care who wrote the schema. That turns out to be exactly the problem, again.

Why This Chapter Exists

Picking up directly from Chapter 1's Comment model prompt — remember the missing index?

- Chapter 1: the AI correctly modeled two relations, missed one index
- Session 3's Generate-Audit-Fix-Retest loop applies here too — just to schemas, not UI
- A schema migration is closer to irreversible than almost anything else an AI touches
- Session 1's rule still holds: specify, verify, override — never just accept
- This chapter is that loop, aimed at indexing, cardinality, and types

> *Nothing new is being taught here — the same verification discipline from Sessions 1 and 3, applied to a new artifact.*

Common AI Schema Mistakes

Modeling

- Many-to-many modeled as two one-to-many (duplication)
- Missing @unique on fields that should be unique
- Optional fields that should be required, or vice versa

Operational

- No index on frequently-filtered foreign keys
- Default CASCADE delete where RESTRICT was intended
- Float used for currency instead of Decimal

These mistakes compile, pass a demo, and only surface as production incidents — under concurrent writes or at scale.

Back to PawMatch

Same app as before — puppies, applications, favorites. Today, one new feature.

- PawMatch wants to add "Foster History"
- Before adoption, many puppies stay with one or more foster families
- The team wants to show each puppy's foster history on their profile
- This means a new table, a new relation, a new migration — a job for a schema

> *A new feature almost always means a new piece of database schema — and that's exactly the moment an AI agent gets asked to help.*

What Is a "Database Schema," Exactly?

In Plain Words

- A schema is the blueprint for how your data is organized
- Which tables exist (Puppy, User, Application...)
- What fields each table has (name, age, status...)
- How tables relate to each other (a Puppy has many Applications)

Think of it as the floor plan of a house — decided before the furniture (your actual data) moves in.

What Does "Good" Even Mean for a Schema?

A schema can be totally valid — and still be a bad idea.

"It Works" Means

- It compiles
- The migration runs without an error
- You can save and read data through it

"It's Good" Also Means

- It models the real relationships correctly
- Common queries are fast, not just possible
- The types match what the data actually is

> *A schema that "works" today can still be the exact thing that breaks your app at scale six months from now.*

Why Ask an AI to Draft It At All?

A fair question, before we hand this job over.

The Honest Upside

- Drafting a first version by hand takes real time
- An AI agent can produce a reasonable starting point in seconds
- It's genuinely good at the mechanical, repetitive parts

The Honest Limit

- It has never seen how your specific app actually queries data
- It can't feel the pain of a slow query at 2am — you can
- "Reasonable starting point" is not the same as "correct and finished"

> *The right way to think about this: the AI is a fast first draft, not a decision-maker.*

Step 1 — Write the Prompt

What we actually typed, asking for the Foster History feature.

prompt to Claude Code

```
Add foster history tracking to the schema. Each puppy
can stay with a foster home before being adopted, and
we want to record when each stay started and ended,
plus the monthly stipend paid to the foster family.
```

AI Helps Here

This is a reasonable first prompt — it states the feature clearly.

You Verify This

Notice what it doesn't say yet: nothing about how many stays a puppy can have, which fields need fast lookups, or what type money should be. We'll come back to all of this.

Step 2 — What the AI Actually Produced

This is the real output — read it the way you would on a real project, before we point anything out.

prisma/schema.prisma (as generated)

```
model FosterHome {
  id        String   @id @default(cuid())
  name      String
  puppyId   String   @unique
  startDate String
  endDate   String
  stipend   Float
  puppy     Puppy    @relation(fields: [puppyId], references: [id])
}
```

It compiles. It reads clearly. If you skimmed this in a busy PR review, would you have caught anything?

What Is "Cardinality"?

Our next term — the first mistake is impossible to explain without it.

In Plain Words

- Cardinality just means: how many of one thing relate to how many of another
- One-to-one: each Puppy has exactly one FosterHome, ever
- One-to-many: each Puppy can have many FosterHome stays, over time
- The wrong choice here isn't a typo — it's a design decision, made wrong

Get cardinality wrong, and the schema doesn't just have a bug — it can't represent reality at all.

MISTAKE 1

The Hidden Mistake: Wrong Cardinality

Look back at Step 2's schema — specifically, the puppyId field.

What the Schema Says

- puppyId String @unique
- @unique means: this value can only appear once, ever
- In practice: each puppy can only ever have ONE foster stay, forever

What the Feature Actually Needs

- Our own prompt said puppies "stay with a foster home" — plural, over time
- A puppy might be fostered twice, or three times, before adoption
- The real relationship is one-to-many, not one-to-one

The AI didn't misunderstand English — it just defaulted to the simpler relationship, because our prompt didn't rule it out.

What Is an "Index"?

Another term worth defining properly before the next mistake makes sense.

In Plain Words

- An index is like the index at the back of a textbook
- Without it, finding "all foster stays for Puppy #42" means checking every single row
- With it, the database jumps straight to the matching rows
- The bigger the table gets, the more this difference matters

A table with ten rows doesn't need an index to feel fast. A table with ten million rows absolutely does.

What a Missing Index Actually Costs

The exact same question — "show Biscuit's foster history" — asked two different ways.

EXPLAIN ANALYZE — FosterHome WHERE puppyId = ?

Without an index

```
Seq Scan on "FosterHome"
  Rows removed: 240,118
Execution Time: 340.2 ms
```

With an index

```
Index Scan using FosterHome_puppyId_idx
  Rows: 2
Execution Time: 1.8 ms
```

Same question, same data — one version checks every row in the table, the other checks two.

The Mistake: No Index for the Common Query

What We'll Actually Do Constantly

- "Show me this puppy's foster history" — filtering by puppyId
- This is the single most common way this table will ever be queried

What the Schema Provides

- No @@index([puppyId]) anywhere in the model
- Every one of those lookups becomes a full table scan
- Invisible with 10 test rows, very visible with 100,000 real ones

The AI wasn't wrong about the data — it just never asked the question "how will this actually be queried?" Nobody told it to.

MISTAKE 3

The Hidden Mistake: Dates Stored as Text

One more look at Step 2 — the startDate and endDate fields.

What the Schema Says

- startDate String
- endDate String
- "2026-03-01" is stored as plain text, not as an actual date

Why That's a Problem

- The database can't sort these correctly (text-sorts "2" before "10")
- "Show stays longer than 2 weeks" becomes very hard to ask
- Nothing stops someone from saving "March 1st" in one row and "03/01" in another

A date that's stored as text looks fine in a demo and quietly breaks the first time anyone tries to sort or filter by it.

MISTAKE 4

The Hidden Mistake: Money Stored as Float

One last look at Step 2 — the stipend field, easy to miss because it looks perfectly reasonable.

What the Schema Says

- stipend Float
- Float is a "floating point" number — the same type used for measurements, scores, coordinates

Why That's a Problem

- Floats can't represent most decimal money amounts exactly
- $10.10 might get stored as 10.099999999999998, quietly
- Add up enough stipend payments and the totals stop matching, by a few cents, for no obvious reason

This is the single most common type mistake in schemas — general-purpose enough to feel safe, wrong specifically for currency.

What Should Money Be Stored As Instead?

The fix, in plain terms, before we see it in code.

Use Decimal, Not Float

- Decimal is a type built specifically to represent exact numbers
- $10.10 stays exactly $10.10 — no silent rounding drift
- Prisma supports it directly: Decimal @db.Decimal(10, 2)
- The (10, 2) means: up to 10 digits total, 2 of them after the decimal point

If a number represents money, the type should always be Decimal — this is close to a universal rule, not a judgment call.

Notice What All Four Mistakes Have in Common

Worth pausing on this before we fix anything.

None of Them Are Typos

- Wrong cardinality: a reasonable-sounding default, not a spelling mistake
- Missing index: something never added, not something written wrong
- String dates: valid Prisma syntax, just the wrong choice of type
- Float currency: also valid syntax, also the wrong choice of type

A schema with real, working syntax can still be wrong in ways that spell-check, a linter, or even a successful migration will never catch.

THE FIX

Step 3 — A Review Checklist for Any AI-Drafted Schema

The four questions that would have caught everything we just found.

Ask These Four Questions

- 1. Could this "one" actually be a "many," over time?
- 2. Which field will this table be searched by most — does it have an index?
- 3. Is every date an actual date type, not a string?
- 4. Is every currency amount a Decimal, not a Float?

These four questions, asked every time, would have caught all four mistakes in this deck without needing to be a database expert.

Step 4 — The Corrected Schema

Each highlighted section fixes one of the four mistakes, in the same order we found them.

prisma/schema.prisma (corrected)

```
model FosterHome {
  id          String    @id @default(cuid())
  name        String
  stays       FosterStay[]
}

model FosterStay {
  id          String    @id @default(cuid())
  puppyId     String
  fosterHomeId String
  startDate   DateTime
  endDate     DateTime?
  stipend     Decimal   @db.Decimal(10, 2)
  puppy       Puppy     @relation(fields: [puppyId], references: [id])
  fosterHome  FosterHome @relation(fields: [fosterHomeId], references: [id])

  @@index([puppyId])
}
```

1. A new FosterStay model — one row per stay, so a puppy can now have many of them. Fixes Mistake 1.

2. @@index([puppyId]) — the common lookup is now fast. Fixes Mistake 2.

3. DateTime instead of String — real dates, sortable and comparable. Fixes Mistake 3.

4. Decimal instead of Float — exact currency, no silent rounding. Fixes Mistake 4.

Four small, specific changes — not a rewrite. Good review rarely means starting over.

Step 5 — Read the Migration Before You Trust It

The schema is only half the story — this is what actually runs against the real database.

prisma/migrations/20260808_add_foster_stay/migration.sql

```
CREATE TABLE "FosterStay" (
  "id" TEXT NOT NULL,
  "puppyId" TEXT NOT NULL,
  "fosterHomeId" TEXT NOT NULL,
  "startDate" TIMESTAMP(3) NOT NULL,
  "endDate" TIMESTAMP(3),
  CONSTRAINT "FosterStay_pkey" PRIMARY KEY ("id")
);

CREATE INDEX "FosterStay_puppyId_idx" ON "FosterStay"("puppyId");
```

AI Helps Here

Prompt: "Run prisma migrate dev and show me the generated migration.sql before I approve it."

You Verify This

Confirm the index is actually in the SQL, not just the schema file — this is the file that actually touches production.

The schema is what you write. The migration.sql is what happens. Reading only one of the two is reading half the story.

THE FOLLY

"It compiled. It migrated without an error. It must be fine."

All three mistakes in this deck passed that exact test — a clean compile catches syntax, never judgment.

The Second Folly — Approving on Vibes

A subtler trap than "it compiled," and just as common.

What It Looks Like

- Skimming the schema, thinking "yeah, that looks about right"
- The field names are sensible, the syntax is clean — nothing jumps out
- Approving because nothing felt wrong, not because you checked something specific

The Fix

- Go back to the three-question checklist, every time
- Ask the cardinality question out loud, not just silently
- "Looks right" is a feeling. "I checked the index" is a fact.

Every mistake in this deck would have survived a "looks fine to me" skim — none of them survive the three-question checklist.

Step 1, Revisited — A Stronger Prompt

Same feature request, now written with everything we learned baked in.

What We Wrote the First Time

- "Add foster history tracking to the schema"
- "Each puppy can stay with a foster home"
- "Record when each stay started and ended, plus a monthly stipend"

What We'd Add Now

- "A puppy may have MULTIPLE foster stays over time"
- "Add an index on whatever field looks up a puppy's stays"
- "Use real date types, not strings, for start and end"
- "Use Decimal, not Float, for the stipend amount"

> *A stronger prompt doesn't guarantee a perfect schema — but every constraint you state in advance is one less thing you have to catch in review.*

The Stronger Prompt, Written Out

prompt to Claude Code (v2)

```
Add foster history tracking. A puppy may have MULTIPLE
foster stays over time (one-to-many, not one-to-one).
Track which foster home, start/end dates using real date
types, and a monthly stipend using Decimal, not Float.
Add an index on the puppy lookup field, since "show this
puppy's foster history" will be a common query.
```

AI Helps Here

This prompt states cardinality, types, currency handling, and indexing explicitly — the exact four things the checklist checks for.

You Verify This

A stronger prompt reduces what review needs to catch. It does not remove the need to review — always re-run the checklist anyway.

When Is an AI-Drafted Schema Safe to Approve?

?	Every relation's cardinality checked against the real feature?	→ **Required**
?	Every commonly-queried field has an index?	→ **Required**
?	Every date stored as a real date type, not a string?	→ **Required**
?	Every currency amount stored as Decimal, not Float?	→ **Required**
?	It compiles and the migration ran with no errors?	→ **Necessary, but not sufficient on its own**

THE ONE RULE

"A schema mistake doesn't show up in a code review the way a UI bug does — it shows up as a slow query or corrupted data, months later, after real information already lives inside it."

That's the entire reason this review discipline exists.

Chapter 4 Key Terms

Background material worth keeping.

Terms

- Cardinality — the shape of a relation: one-to-one, one-to-many, many-to-many
- Index — a database structure that turns a table scan into a fast lookup
- EXPLAIN ANALYZE — Postgres's tool for showing a query's actual execution plan
- Enum — a type restricted to a fixed set of values, enforced by the database

END OF CHAPTER 4

AI-Assisted Schema Design

- ✓ AI schema drafts get syntax right but often miss indexing, cardinality, and constraints
- ✓ Missing indexes and Float-for-currency are the most common costly mistakes
- ✓ Review the schema diff AND the generated SQL — never run migrate deploy unread
- ✓ Prompt with explicit cardinality, indexing, and constraint requirements

The takeaway: *A migration is close to irreversible in production — it deserves more scrutiny than any other AI-generated artifact in this course.*

Final Story, Told in Four Chapters

- ~~Prisma ORM — schema, migrations, CRUD~~
- ~~NextAuth.js OAuth — protected routes~~
- ~~TanStack Query — server state, cache invalidation~~
- ~~AI-assisted schema design — prompting and verifying AI migrations~~

Up next

The hands-on lab — an AI-drafted schema, reviewed, migrated, and wired to a real protected, data-fetching route.